

SSSSSSSS

EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. Designed for students of all academic levels, EduGenie enables users to:

- Ask questions and receive smart, concise answers
- Understand complex concepts through simplified explanations
- Generate quizzes from topics or text
- Receive personalized learning recommendations
- Summarize large educational passages

Built with **FastAPI** for the backend and a simple **HTML+CSS frontend**, EduGenie leverages lightweight and cloud-based AI models for local efficiency and cloud power. It works well on devices like the Mac M1, making it accessible to a broad range of learners and developers .

Scenarios

Scenario 1: A student wants to know about oceans and rivers uses EduGenie to ask “Which is the largest ocean?”

Scenario 2: A student wants to know the level of her understanding of “The Pythagoras Theorem” and clicks “Generate Quiz.”

Scenario 3: A learner exploring SQL requests a learning pat

1. 5. Click ****OK**** to save. Open a new terminal session to load the variable.
- 2.
3. **### Option 2: Provide the API key explicitly in code**
- 4.
5. You can pass the API key explicitly when initializing the client. Only do this

```
6. if you cannot use environment variables.
7.
8. ### Python
9.
10. from google import genai
11.
12. client = genai.Client(api_key="YOUR_API_KEY")
13.
14. interaction = client.interactions.create(
15.     model="gemini-3.8-flash",
16.     input="Explain how AI works in a few words"
17. )
18. print(interaction.output_text)
19.
20. ### JavaScript
21.
22. import { GoogleGenAI } from "@google/genai";
23.
24. const ai = new GoogleGenAI({ apiKey: "YOUR_API_KEY"
    });
25.
26. async function main() {
27.     const interaction = await ai.interactions.create({
28.         model: "gemini-3.8-flash",
29.         input: "Explain how AI works in a few words",
30.     });
31.     console.log(interaction.output_text);
32. }
```

```
33.  
34.  main();  
35.  
36. ### Java  
37.  
38.  import com.google.genai.Client;  
39.  import  
    com.google.genai.gaos.models.interactions.CreateModelInteraction;  
40.  import  
    com.google.genai.gaos.models.interactions.Interaction;  
41.  import  
    com.google.genai.gaos.models.interactions.InteractionsInput;  
42.  import  
    com.google.genai.gaos.models.interactions.Model;  
43.  import  
    com.google.genai.gaos.models.operations.CreateInteractionRequestBody;  
44.  
45.  Client client =  
    Client.builder().apiKey("YOUR_API_KEY").build();  
46.  
47.  CreateModelInteraction params =  
48.      CreateModelInteraction.builder()  
49.          .model(Model.of("gemini-3.8-flash"))  
50.          .input(InteractionsInput.of("Explain how AI works in  
    a few sentences."))  
51.          .build();  
52.
```

```
53. Interaction interaction =
54.
    client.interactions.create(CreateInteractionRequestBody.o
    f(params)).interaction().get();
55.
56. System.out.println(interaction.outputText().orElse(""));
57.
58. ### REST
59.
60. curl
    "https://generativelanguage.googleapis.com/v1beta/inter
    actions" \
61.     -H 'Content-Type: application/json' \
62.     -H "x-goog-api-key: YOUR_API_KEY" \
63.     -X POST \
64.     -d '{
65.         "model": "gemini-3.8-flash",
66.         "input": "Explain how AI works in a few words"
67.     }'
68.
69. ## Security and secret management
70.
71. Treat your Gemini API key like a password. If
    compromised, others can consume
72. your project's quota, incur unexpected billing charges, and
    access private
73. resources.
74.
75. ### Critical security rules
```

76.

77. - **Keep keys confidential**: Never check API keys into source control systems like Git.

78. - **Never expose keys client-side in production**: Do not hardcode API keys directly in web or mobile apps. Keys compiled in client-side code can be extracted by users. To secure client-side apps, run a backend proxy server to make the actual API calls.

79.

80. **Secret management best practices**

81.

82. - **Environment variables**: Read keys from environment variables rather than configuration files.

83. - **Secret Manager**: For production, store your keys in a secure secret store such as [Google Cloud Secret Manager](<https://cloud.google.com/secret-manager>).

84. - **Billing alerts**: Set up billing alerts in the Google Cloud Console to notify you if usage or costs spike.

85.

86. **Leak response checklist**

87.

88. If you suspect your API key has been leaked:

89.

90. 1. **Generate a new key**: Create a replacement key in Google AI Studio or the Cloud Console.

91. 2. **Update your application**: Deploy your code using the new key.

92. 3. **Disable or delete the compromised key**: Disable the leaked key in the Cloud Console once the new key is verified. Do not delete the old key until the new key is fully active to avoid application downtime.

93. 4. **Audit usage**: Check billing logs and API usage in the Google Cloud Console to identify unauthorized activity.

94.

95. **Restricting and securing your keys**

96.

97. Adding restrictions to your API keys minimizes the potential damage if a key is

98. compromised.

99.

100. **Apply request origin restrictions**

101.

102. Origin restrictions limit which IP addresses, websites, or applications can use

103. your key.

104.

105. 1. Go to the [Google Cloud Console Credentials page](<https://console.cloud.google.com/apis/credentials>).

106. 2. Select your project, and click the name of the API key you want to restrict.

107. 3. Under **Application restrictions** , select **IP addresses** (or the appropriate restriction type for your environment).

108. 4. Specify the allowed IP addresses or ranges, then click **Save**.

109.

110. **Securing unrestricted standard API keys**

111.

112. To continue using the Gemini API, you must secure any unrestricted keys.

113.

114. ##### Method A: Restrict the key to the Gemini API only
(AI Studio)
- 115.
116. If you only use the key for the Gemini API, secure it
directly in AI Studio:
- 117.
118. 1. On the ****API Keys**** page in [Google AI
Studio](<https://aistudio.google.com/api-keys>), locate keys
marked with the ****Unrestricted**** label.
119. 2. Hover over the label and click ****Add restrictions**** in
the dialog.
120. 3. Select ****Restrict to Gemini API only****.
121. 4. Click ****Restrict key**** to confirm.
- 122.
123. > [!NOTE]
124. > ****Note:**** To restrict an API key, you must have the
'apikey.keys.update' permission on the associated
Google Cloud project. This permission is included in roles
like ****API Keys Admin**** or ****Editor****.
- 125.
126. ##### Method B: Restrict the key for other services
(Google Cloud Console)
- 127.
128. If the key is shared with other Google APIs (not
recommended), restrict it in
129. the Cloud Console. ****Note:** Gemini API requests using
this key will fail after
130. these restrictions are applied.**
- 131.
132. 1. Visit the [Google Cloud Console Credentials
page](<https://console.cloud.google.com/apis/credentials>).

133. 2. Select the project and the API key.
134. 3. Under ****API restrictions**** , use the ****Select API restrictions**** drop-down to select the APIs you want this key to access. Do not select the ****Generative**
135. **Language API****.
136. 4. Click ****Save****. Create a separate, restricted key in AI Studio to continue using the Gemini API.
- 137.
138. **### Blocked dormant keys**
- 139.
140. Starting May 7, 2026, the Gemini API blocks unrestricted API keys that have
141. been dormant for an extended period. These keys show a ****Blocked**** tag in AI
142. Studio. You must generate a new key or use an existing restricted key to
143. continue.
- 144.
145. **## Migrate to an auth key**
- 146.
147. Follow these steps to create a new auth API key and update your applications:
- 148.
149. 1. Go to the [AI Studio API Keys page](https://aistudio.google.com/api-keys).
150. 2. Check the ****Key Type**** column to identify any keys listed as ****Standard****.
151. 3. Click ****Create API key**** to generate a new key. All new keys created in AI Studio are automatically created as auth keys.
152. 4. Copy the new auth API key.

153. 5. Update your application code, environment variables, and any deployment configurations to use the new auth API key.
154. 6. Test your application to confirm it works correctly with the new key.
155. 7. Once verified, delete or revoke your old traffic key to prevent misuse.
- 156.
157. > [!NOTE]
158. > **Note:** If you don't see your keys, you may need to import your Cloud projects into AI Studio. Refer to the [import projects](<https://ai.google.dev/gemini-api/docs/api-key#import-projects>) section for instructions.
- 159.
160. ## Limitations
- 161.
162. Google AI Studio imposes the following project and key management limitations:
- 163.
164. - You can create a maximum of 10 projects at a time from the Google AI Studio **Projects** page.
165. - The **API keys** and **Projects** pages display a maximum of 100 keys and 50 projects.
166. - Only API keys that are unrestricted or restricted specifically to the Generative Language API (Gemini API) are displayed.
- 167.
168. For advanced project management or to modify keys with other restrictions, use
169. the [Google Cloud Console credentials page](<https://console.cloud.google.com/apis/credentials>).

170. To use the Gemini API, you must authenticate your requests. You can
171. authenticate using a standard or authorization API key.
- 172.
173. [Create or view a Gemini API Key](<https://aistudio.google.com/apikey>)
- 174.
175. ## API key types: standard versus authorization
- 176.
177. API keys provide access to the Gemini API, but their security characteristics
178. differ. The Gemini API is transitioning from standard API keys to authorization
179. keys to improve security:
- 180.
181. - **Standard API keys**: Associate requests with a Google Cloud project for billing and quota purposes. Standard keys don't identify a caller, which limits the granularity of permissions and access control they can support.
182. - **Authorization (auth) keys**: Bound directly to a Google Cloud service account. When you use an authorization key, your requests are processed under the identity of that bound service account, enabling granular access control. Authorization keys are restricted to the Generative Language API (Gemini API) by default and provide fast-acting leaked key enforcement that quickly stops the usage of leaked keys detected by our systems.
- 183.
184. To ensure secure usage, Gemini API will move from standard keys to auth keys:
- 185.

186. - **Auth keys default**: Starting May 28, 2026, all new API keys created in Google AI Studio are automatically created as auth keys.
187. - **Unrestricted keys rejected** : The Gemini API rejects requests from **unrestricted standard keys**. Standard API keys that have explicit restrictions applied continue to work. This restriction prevents the unauthorized use of keys that might be shared publicly or linked to other services.
188. - **On September 2026** : the Gemini API will reject requests from **standard**
189. **keys** . You must [migrate to auth keys](<https://ai.google.dev/gemini-api/docs/api-key#migrate-to-auth-key>) before this date to avoid service interruption. Make sure to migrate to auth keys before September 2026.
- 190.
191. > [!NOTE]
192. > **Note**: Requests authenticated by authorization keys are not recorded in Google Cloud service account usage metrics.
- 193.
194. **## Managing API keys in Google AI Studio**
- 195.
196. You can manage your projects and keys directly in [Google AI Studio](<https://aistudio.google.com/apikey>).
- 197.
198. **### Google Cloud projects**
- 199.
200. Every Gemini API key is associated with a [Google Cloud project](<https://cloud.google.com/resource-manager/docs/creating-managing-projects>).

201. Google Cloud projects manage billing, collaborators, and permissions. Google AI
202. Studio provides a lightweight interface to access these projects.
- 203.
204. - **Default project** : If you are a new user, Google AI Studio automatically creates a default Google Cloud project and API key after you accept the Terms of Service. You can rename this project by navigating to the **Projects** view in your dashboard.
205. - **Existing projects**: If you already have a Google Cloud account, AI Studio does not create a default project. Instead, you must import your existing projects.
- 206.
207. ### Importing projects
- 208.
209. By default, Google AI Studio does not display all of your Google Cloud
210. projects. You must import the projects you want to use:
- 211.
212. 1. Go to [Google AI Studio](https://aistudio.google.com).
213. 2. Open the **Dashboard** from the left panel and select **Projects**.
214. 3. Click the **Import projects** button.
215. 4. Search for and select the Google Cloud project you want to import, then click **Import**.
216. 5. Once imported, navigate to the **API Keys** page in the dashboard to create a key in that project.
- 217.
218. ### Troubleshooting key creation permissions

- 219.
220. If the ****Create API key**** button is unavailable and displays the message:
221. **"You do not have permission to create a key in this project"**, you lack the
222. required IAM permissions.
- 223.
224. Ask your Google Cloud project or organization administrator to grant you a role
225. containing the following permissions (such as Project Editor):
- 226.
227. - ``resourcemanager.projects.get``: Allows AI Studio to verify the project.
228. - ``apikeys.keys.create``: Allows key generation.
229. - ``serviceusage.services.enable``: Ensures the Generative Language API is enabled.
230. - ``iam.serviceAccounts.create``: Required to create the linked service account.
231. - ``iam.serviceAccountApiKeyBindings.create``: Binds the service account to the API key.
- 232.
233. If you cannot get administrative access, you can create a new Google Cloud
234. project that is not associated with an organization to generate your keys.
- 235.
236. **## Setting up your environment**
- 237.
238. Once you have a key, configure your environment to use it securely in your

- 239. applications.
- 240.
- 241. `### Option 1: Use environment variables`
(recommended)
- 242.
- 243. Set the environment variable ``GEMINI_API_KEY`` or ``GOOGLE_API_KEY``. The Gemini
- 244. API client libraries automatically detect and use these variables. If both are
- 245. set, ``GOOGLE_API_KEY`` takes precedence.
- 246.
- 247. Select your operating system to set the variable:
- 248.
- 249. `### Linux/macOS - Bash`
- 250.
- 251. Verify if you have a bash configuration file:
- 252.
- 253. `~/.bashrc`
- 254.
- 255. If not, create one and open it:
- 256.
- 257. `touch ~/.bashrc && open ~/.bashrc`
- 258.
- 259. Add the export command at the end of the file:
- 260.
- 261. `export GEMINI_API_KEY=<YOUR_API_KEY_HERE>`
- 262.
- 263. Save the file, then apply the changes:

264.

265. `source ~/.bashrc`

266.

267. `### macOS - Zsh`

268.

269. Verify if you have a zsh configuration file:

270.

271. `~/.zshrc`

272.

273. If not, create one and open it:

274.

275. `touch ~/.zshrc && open ~/.zshrc`

276.

277. Add the export command:

278.

279. `export GEMINI_API_KEY=<YOUR_API_KEY_HERE>`

280.

281. Save the file, then apply the changes:

282.

283. `source ~/.zshrc`

284.

285. `### Windows`

286.

287. 1. Search for "Environment Variables" in the Windows search bar.

288. 2. Click ****Environment Variables**** in the System Properties dialog.

289. 3. Under ****User variables**** or ****System variables**** ,
click ****New...****.
290. 4. Set the variable name to ``GEMINI_API_KEY`` and the
value to your API key.
291. 5. Click ****OK**** to save. Open a new terminal session to
load the variable.
- 292.
293. **### Option 2: Provide the API key explicitly in code**
- 294.
295. You can pass the API key explicitly when initializing the
client. Only do this
296. if you cannot use environment variables.
- 297.
298. **### Python**
- 299.
300. `from google import genai`
- 301.
302. `client = genai.Client(api_key="YOUR_API_KEY")`
- 303.
304. `interaction = client.interactions.create(`
305. `model="gemini-3.8-flash",`
306. `input="Explain how AI works in a few words"`
307. `)`
308. `print(interaction.output_text)`
- 309.
310. **### JavaScript**
- 311.
312. `import { GoogleGenAI } from "@google/genai";`

313.

```
314.   const ai = new GoogleGenAI({ apiKey:  
      "YOUR_API_KEY" });
```

315.

```
316.   async function main() {  
317.     const interaction = await ai.interactions.create({  
318.       model: "gemini-3.8-flash",  
319.       input: "Explain how AI works in a few words",  
320.     });  
321.     console.log(interaction.output_text);  
322.   }
```

323.

```
324.   main();
```

325.

```
326.   ### Java
```

327.

```
328.   import com.google.genai.Client;  
329.   import  
      com.google.genai.gaos.models.interactions.CreateModelI  
      nteraction;  
330.   import  
      com.google.genai.gaos.models.interactions.Interaction;  
331.   import  
      com.google.genai.gaos.models.interactions.InteractionsIn  
      put;  
332.   import  
      com.google.genai.gaos.models.interactions.Model;  
333.   import  
      com.google.genai.gaos.models.operations.CreateInteracti  
      onRequestBody;
```

```
334.
335.     Client client =
        Client.builder().apiKey("YOUR_API_KEY").build();
336.
337.     CreateModelInteraction params =
338.         CreateModelInteraction.builder()
339.             .model(Model.of("gemini-3.8-flash"))
340.             .input(InteractionsInput.of("Explain how AI works
                in a few sentences."))
341.             .build();
342.
343.     Interaction interaction =
344.         client.interactions.create(CreateInteractionRequestBody.o
            f(params)).interaction().get();
345.
346.     System.out.println(interaction.outputText().orElse(""));
347.
348. ### RESTTo use the Gemini API, you must authenticate
        your requests. You can
349. authenticate using a standard or authorization API key.
350.
351. [Create or view a Gemini API
        Key](https://aistudio.google.com/apikey)
352.
353. ## API key types: standard versus authorization
354.
355. API keys provide access to the Gemini API, but their
        security characteristics
```

356. differ. The Gemini API is transitioning from standard API keys to authorization
357. keys to improve security:
- 358.
359. - **Standard API keys**: Associate requests with a Google Cloud project for billing and quota purposes. Standard keys don't identify a caller, which limits the granularity of permissions and access control they can support.
360. - **Authorization (auth) keys**: Bound directly to a Google Cloud service account. When you use an authorization key, your requests are processed under the identity of that bound service account, enabling granular access control. Authorization keys are restricted to the Generative Language API (Gemini API) by default and provide fast-acting leaked key enforcement that quickly stops the usage of leaked keys detected by our systems.
- 361.
362. To ensure secure usage, Gemini API will move from standard keys to auth keys:
- 363.
364. - **Auth keys default**: Starting May 28, 2026, all new API keys created in Google AI Studio are automatically created as auth keys.
365. - **Unrestricted keys rejected** : The Gemini API rejects requests from **unrestricted standard keys**. Standard API keys that have explicit restrictions applied continue to work. This restriction prevents the unauthorized use of keys that might be shared publicly or linked to other services.
366. - **On September 2026** : the Gemini API will reject requests from **standard**
367. **keys** . You must [migrate to auth keys](<https://ai.google.dev/gemini-api/docs/api-key#migrate-to-auth-key>) before this date to avoid service

interruption. Make sure to migrate to auth keys before September 2026.

368.

369. > [!NOTE]

370. > **Note:** Requests authenticated by authorization keys are not recorded in Google Cloud service account usage metrics.

371.

372. ## Managing API keys in Google AI Studio

373.

374. You can manage your projects and keys directly in [Google AI Studio](<https://aistudio.google.com/apikey>).

375.

376. ### Google Cloud projects

377.

378. Every Gemini API key is associated with a [Google Cloud project](<https://cloud.google.com/resource-manager/docs/creating-managing-projects>).

379. Google Cloud projects manage billing, collaborators, and permissions. Google AI

380. Studio provides a lightweight interface to access these projects.

381.

382. - **Default project** : If you are a new user, Google AI Studio automatically creates a default Google Cloud project and API key after you accept the Terms of Service. You can rename this project by navigating to the **Projects** view in your dashboard.

383. - **Existing projects**: If you already have a Google Cloud account, AI Studio does not create a default project. Instead, you must import your existing projects.

384.

385. ### Importing projects

386.

387. By default, Google AI Studio does not display all of your Google Cloud

388. projects. You must import the projects you want to use:

389.

390. 1. Go to [Google AI Studio](<https://aistudio.google.com>).

391. 2. Open the **Dashboard** from the left panel and select **Projects**.

392. 3. Click the **Import projects** button.

393. 4. Search for and select the Google Cloud project you want to import, then click **Import**.

394. 5. Once imported, navigate to the **API Keys** page in the dashboard to create a key in that project.

395.

396. ### Troubleshooting key creation permissions

397.

398. If the **Create API key** button is unavailable and displays the message:

399. **"You do not have permission to create a key in this project"**, you lack the

400. required IAM permissions.

401.

402. Ask your Google Cloud project or organization administrator to grant you a role

403. containing the following permissions (such as Project Editor):

404.

- 405. - ``resourcemanager.projects.get``: Allows AI Studio to verify the project.
- 406. - ``apikeys.keys.create``: Allows key generation.
- 407. - ``serviceusage.services.enable``: Ensures the Generative Language API is enabled.
- 408. - ``iam.serviceAccounts.create``: Required to create the linked service account.
- 409. - ``iam.serviceAccountApiKeyBindings.create``: Binds the service account to the API key.
- 410.
- 411. If you cannot get administrative access, you can create a new Google Cloud
- 412. project that is not associated with an organization to generate your keys.
- 413.
- 414. ## Setting up your environment
- 415.
- 416. Once you have a key, configure your environment to use it securely in your
- 417. applications.
- 418.
- 419. ### Option 1: Use environment variables (recommended)
- 420.
- 421. Set the environment variable ``GEMINI_API_KEY`` or ``GOOGLE_API_KEY``. The Gemini
- 422. API client libraries automatically detect and use these variables. If both are
- 423. set, ``GOOGLE_API_KEY`` takes precedence.
- 424.

425. Select your operating system to set the variable:
- 426.
427. `### Linux/macOS - Bash`
- 428.
429. Verify if you have a bash configuration file:
- 430.
431. `~/.bashrc`
- 432.
433. If not, create one and open it:
- 434.
435. `touch ~/.bashrc && open ~/.bashrc`
- 436.
437. Add the export command at the end of the file:
- 438.
439. `export GEMINI_API_KEY=<YOUR_API_KEY_HERE>`
- 440.
441. Save the file, then apply the changes:
- 442.
443. `source ~/.bashrc`
- 444.
445. `### macOS - Zsh`
- 446.
447. Verify if you have a zsh configuration file:
- 448.
449. `~/.zshrc`
- 450.
451. If not, create one and open it:

452.

453. `touch ~/.zshrc && open ~/.zshrc`

454.

455. Add the export command:

456.

457. `export GEMINI_API_KEY=<YOUR_API_KEY_HERE>`

458.

459. Save the file, then apply the changes:

460.

461. `source ~/.zshrc`

462.

463. **### Windows**

464.

465. 1. Search for "Environment Variables" in the Windows search bar.

466. 2. Click **Environment Variables** in the System Properties dialog.

467. 3. Under **User variables** or **System variables**, click **New...**.

468. 4. Set the variable name to `GEMINI_API_KEY` and the value to your API key.

469. 5. Click **OK** to save. Open a new terminal session to load the variable.

470.

471. **### Option 2: Provide the API key explicitly in code**

472.

473. You can pass the API key explicitly when initializing the client. Only do this

474. if you cannot use environment variables.

```
475.
476. ### Python
477.
478.     from google import genai
479.
480.     client = genai.Client(api_key="YOUR_API_KEY")
481.
482.     interaction = client.interactions.create(
483.         model="gemini-3.8-flash",
484.         input="Explain how AI works in a few words"
485.     )
486.     print(interaction.output_text)
487.
488. ### JavaScript
489.
490.     import { GoogleGenAI } from "@google/genai";
491.
492.     const ai = new GoogleGenAI({ apiKey:
493.         "YOUR_API_KEY" });
494.
495.     async function main() {
496.         const interaction = await ai.interactions.create({
497.             model: "gemini-3.8-flash",
498.             input: "Explain how AI works in a few words",
499.         });
500.         console.log(interaction.output_text);
501.     }
```

```
502.     main();
503.
504.     ### Java
505.
506.     import com.google.genai.Client;
507.     import
        com.google.genai.gaos.models.interactions.CreateModelI
        nteraction;
508.     import
        com.google.genai.gaos.models.interactions.Interaction;
509.     import
        com.google.genai.gaos.models.interactions.InteractionsIn
        put;
510.     import
        com.google.genai.gaos.models.interactions.Model;
511.     import
        com.google.genai.gaos.models.operations.CreateInteracti
        onRequestBody;
512.
513.     Client client =
        Client.builder().apiKey("YOUR_API_KEY").build();
514.
515.     CreateModelInteraction params =
516.         CreateModelInteraction.builder()
517.             .model(Model.of("gemini-3.8-flash"))
518.             .input(InteractionsInput.of("Explain how AI works
                in a few sentences."))
519.             .build();
520.
521.     Interaction interaction =
```

```
522.         client.interactions.create(CreateInteractionRequestBody.o
              f(params)).interaction().get();

523.

524.         System.out.println(interaction.outputText().orElse(""));

525.

526. #### REST

527.

528.     curl
        "https://generativelanguage.googleapis.com/v1beta/inter
        actions" \

529.     -H 'Content-Type: application/json' \

530.     -H "x-goog-api-key: YOUR_API_KEY" \

531.     -X POST \

532.     -d '{

533.         "model": "gemini-3.8-flash",

534.         "input": "Explain how AI works in a few words"

535.     }'

536.

537. ## Security and secret management

538.

539. Treat your Gemini API key like a password. If
    compromised, others can consume

540. your project's quota, incur unexpected billing charges,
    and access private

541. resources.

542.

543. #### Critical security rules
```

544.

545. - **Keep keys confidential**: Never check API keys into source control systems like Git.

546. - **Never expose keys client-side in production**: Do not hardcode API keys directly in web or mobile apps. Keys compiled in client-side code can be extracted by users. To secure client-side apps, run a backend proxy server to make the actual API calls.

547.

548. **Secret management best practices**

549.

550. - **Environment variables**: Read keys from environment variables rather than configuration files.

551. - **Secret Manager** : For production, store your keys in a secure secret store such as [Google Cloud Secret Manager](<https://cloud.google.com/secret-manager>).

552. - **Billing alerts**: Set up billing alerts in the Google Cloud Console to notify you if usage or costs spike.

553.

554. **Leak response checklist**

555.

556. If you suspect your API key has been leaked:

557.

558. 1. **Generate a new key**: Create a replacement key in Google AI Studio or the Cloud Console.

559. 2. **Update your application**: Deploy your code using the new key.

560. 3. **Disable or delete the compromised key**: Disable the leaked key in the Cloud Console once the new key is verified. Do not delete the old key until the new key is fully active to avoid application downtime.

561. 4. **Audit usage**: Check billing logs and API usage in the Google Cloud Console to identify unauthorized activity.
- 562.
563. **Restricting and securing your keys**
- 564.
565. Adding restrictions to your API keys minimizes the potential damage if a key is
566. compromised.
- 567.
568. **Apply request origin restrictions**
- 569.
570. Origin restrictions limit which IP addresses, websites, or applications can use
571. your key.
- 572.
573. 1. Go to the [Google Cloud Console Credentials page](<https://console.cloud.google.com/apis/credentials>).
574. 2. Select your project, and click the name of the API key you want to restrict.
575. 3. Under **Application restrictions** , select **IP addresses** (or the appropriate restriction type for your environment).
576. 4. Specify the allowed IP addresses or ranges, then click **Save**.
- 577.
578. **Securing unrestricted standard API keys**
- 579.
580. To continue using the Gemini API, you must secure any unrestricted keys.

581.

582. ##### Method A: Restrict the key to the Gemini API only
(AI Studio)

583.

584. If you only use the key for the Gemini API, secure it
directly in AI Studio:

585.

586. 1. On the **API Keys** page in [Google AI
Studio](<https://aistudio.google.com/api-keys>), locate keys
marked with the **Unrestricted** label.

587. 2. Hover over the label and click **Add restrictions** in
the dialog.

588. 3. Select **Restrict to Gemini API only**.

589. 4. Click **Restrict key** to confirm.

590.

591. > [!NOTE]

592. > **Note:** To restrict an API key, you must have the
'apikeykeys.update' permission on the associated
Google Cloud project. This permission is included in roles
like **API Keys Admin** or **Editor**.

593.

594. ##### Method B: Restrict the key for other services
(Google Cloud Console)

595.

596. If the key is shared with other Google APIs (not
recommended), restrict it in

597. the Cloud Console. **Note:** Gemini API requests using
this key will fail after

598. these restrictions are applied.

599.

600. 1. Visit the [Google Cloud Console Credentials page](https://console.cloud.google.com/apis/credentials).
601. 2. Select the project and the API key.
602. 3. Under **API restrictions** , use the **Select API restrictions** drop-down to select the APIs you want this key to access. Do not select the **Generative Language API**.
603. 4. Click **Save**. Create a separate, restricted key in AI Studio to continue using the Gemini API.
- 604.
- 605.
606. ### Blocked dormant keys
- 607.
608. Starting May 7, 2026, the Gemini API blocks unrestricted API keys that have
609. been dormant for an extended period. These keys show a **Blocked** tag in AI
610. Studio. You must generate a new key or use an existing restricted key to
611. continue.
- 612.
613. ## Migrate to an auth key
- 614.
615. Follow these steps to create a new auth API key and update your applications:
- 616.
617. 1. Go to the [AI Studio API Keys page](https://aistudio.google.com/api-keys).
618. 2. Check the **Key Type** column to identify any keys listed as **Standard**.

619. 3. Click **Create API key** to generate a new key. All new keys created in AI Studio are automatically created as auth keys.
620. 4. Copy the new auth API key.
621. 5. Update your application code, environment variables, and any deployment configurations to use the new auth API key.
622. 6. Test your application to confirm it works correctly with the new key.
623. 7. Once verified, delete or revoke your old traffic key to prevent misuse.
- 624.
625. > [!NOTE]
626. > **Note:** If you don't see your keys, you may need to import your Cloud projects into AI Studio. Refer to the [import projects](https://ai.google.dev/gemini-api/docs/api-key#import-projects) section for instructions.
- 627.
628. ## Limitations
- 629.
630. Google AI Studio imposes the following project and key management limitations:
- 631.
632. - You can create a maximum of 10 projects at a time from the Google AI Studio **Projects** page.
633. - The **API keys** and **Projects** pages display a maximum of 100 keys and 50 projects.
634. - Only API keys that are unrestricted or restricted specifically to the Generative Language API (Gemini API) are displayed.
- 635.

```
636. For advanced project management or to modify keys
    with other restrictions, use

637. the [Google Cloud Console credentials
    page](https://console.cloud.google.com/apis/credentials).

638.

639.

640. curl
    "https://generativelanguage.googleapis.com/v1beta/inter
    actions" \

641.     -H 'Content-Type: application/json' \

642.     -H "x-goog-api-key: YOUR_API_KEY" \

643.     -X POST \

644.     -d '{

645.         "model": "gemini-3.8-flash",

646.         "input": "Explain how AI works in a few words"

647.     }'

648.

    • sooo good
    •
    • Popular Tutorial: Python Programming Tutorial - Full Course for Beginners
    • Installation Steps:

        1. Download the latest Python 3.10+ installer for
            your operating system from the official website.
        2. Run the installer and ensure you check the box
            that says "Add Python to PATH".
        3. Follow the installation prompts to complete the
            setup.
        4. Verify the installation by opening a terminal or
            command prompt and typing python --version.
```

2. FastAPI Framework –

- Official Documentation: [FastAPI](#)
- User Guide: [FastAPI](#)
- Popular Tutorial: [FastAPI Crash Course](#)

3. HTML & CSS – Basic templating used in /templates and /static

4. Google Gemini API Key

- Official Documentation: [Google AI for Developers](#)
- Setup Guide: [GeeksforGeeks](#)
- Popular Tutorial: [How to Use Google Gemini API Key](#)
- Setup Steps:
 1. Visit the Google AI Studio and sign in with your Google account.
 2. Create a new project and enable the Gemini API.
 3. Generate an API key and securely store it.

5. Uvicorn (ASGI Server)

- Official Documentation: [PyPI](#)
- Installation Steps:.....
 1. Install Uvicorn using pip: pip install uvicorn

6. Jinja2 (HTML Templating Engine)

- Official Documentation: Jinja2 Documentation
- Installation Steps:
 1. Install Jinja2 using pip: pip install jinja2

Project Workflow

MILESTONE 1: Model Selection and Architecture

Activity 1.1: Select AI Models

- **Gemini 1.5 Pro (via API):**
Used for: Q&A, summarization, quiz generation, learning paths
Benefits: Advanced reasoning, structured outputs, and cloud inference
- **LaMini-Flan-T5-783M (local):**
Used for: Concept explanation
Benefits: Instruction-tuned, lightweight, CPU-compatible

Folder Architecture:

EduGenie/		
1.	main.py	# FastAPI app
2.	explanation_module.py	# Concept explanation logic
3.	qna.py	# Question answering
4.	quiz_module.py	# Quiz generation
5.	summary_module.py	# Summarization
6.	learning_path.py	# Learning recommendations
7.	templates/index.html	# HTML frontend
8.	static/style.css	# Styling
9.	requirements.txt	# Python dependencies

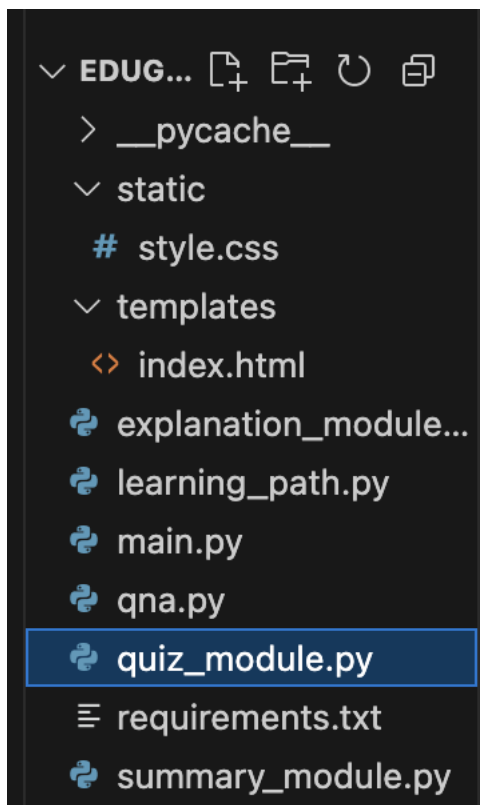


FIG. Showing the Folder Architecture

MILESTONE 2: Core Functionalities Development

Activity 2.1. : Module Implementation

1. Explanation Module

EduGenie utilizes the LaMini-Flan-T5 model, a lightweight yet powerful generative AI, to deliver educational content in a simplified and highly readable manner. This model is specifically fine-tuned to provide concise, context-aware responses that break down complex topics into easily understandable language. By focusing on clarity and brevity, EduGenie ensures that learners with minimal background knowledge or technical experience can grasp essential concepts without feeling overwhelmed. This makes it particularly valuable for beginners, school students, or self-learners seeking straightforward explanations. The integration of LaMini-Flan-T5 empowers EduGenie to act as a reliable and accessible study companion for foundational learning across subjects.

```

planation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
  
```

Fig. Explanation module logic

2. QnA Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

```

def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in QnA: {e}"
  
```

Fig. Q/A Module

3. Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:(?:json)?\n(.*?))```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧹 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
```

Fig. Quiz Module Logic

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the `clean_json_block` function. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

4. Summary Module

This feature leverages Gemini's generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.

```
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Fig. Summary Module

5. Learning Path Module

The `get_learning_recommendations` function uses Google Gemini to generate a personalized, structured learning path for any given topic. By crafting a detailed prompt, it instructs the AI to suggest beginner to advanced concepts, organized by difficulty, and supported with useful resources such as videos, articles, or books. It ensures adaptability to the learner's level. The function safely handles API responses and errors, providing either the generated text or a helpful error message if the response is invalid or incomplete.

```
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (1
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🗨️ Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc() # ✅ This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```

Fig. Learning Path Module

Activity 2.2 : Backend API with FastAPI

- Defined RESTful endpoints for each module:
 - /qa
 - /explain
 - /quiz
 - /summarize
 - /learn/recommendations
- Connected each endpoint to a respective module logic

```

main.py
40
41 # Q&A - GET API using Gemini
42 @app.get("/qa")
43 async def answer_question(question: str = Query(...)):
44     answer = answer_question_with_gemini(question)
45     return {"answer": answer}
46
47 # Explanation - POST API
48 @app.post("/explain/")
49 async def explain_api(request: Request):
50     data = await request.json()
51     topic = data.get("topic")
52     if not topic:
53         return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
54     explanation = explain_topic(topic)
55     return {"topic": topic, "explanation": explanation}
56
57 # Summarization - POST API
58 @app.post("/summarize/")
59 async def summarize_api(request: Request):
60     data = await request.json()
61     text = data.get("text")
62     if not text:
63         return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
64     summary = summarize_text(text)
65     return {"summary": summary}
66
  
```

Fig. Endpoints of API

```

66
67 # Quiz Generation - POST API
68 @app.post("/quiz")
69 async def quiz_api(request: Request):
70     data = await request.json()
71     text = data.get("text")
72     if not text:
73         return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
74     quiz = generate_quiz(text)
75     print("Generated quiz:", quiz) # [X] DEBUG
76     return JSONResponse(content={"quiz": quiz})
77
78 # Learning Recommendations - GET API
79 @app.get("/learn/recommendations")
80 async def learning_recommendation_api(topic: str = Query(...)):
81     recommendation = get_learning_recommendations(topic)
82     return {"topic": topic, "recommendation": recommendation}
83
  
```

Fig. Endpoints of API

MILESTONE 3: Frontend Development

Activity 3.1: Build Web Interface

- HTML Form (index.html):
 - Task dropdown (Explain, QnA, Quiz, Summary, Recommend Path)
 - Text area for user input
 - Submit button
- CSS Styling (style.css):
 - Responsive design
 - Styled buttons, input areas, and result container

```
templates > <> index.html > ...
 2  <html lang="en">
 3  <head>
86  </head>
87  <body>
88    <h1>Welcome to EduGenie 🤖</h1>
89    <p>Your personal AI tutor for learning support!</p>
90
91    <!-- Q&A -->
92    <form id="qaForm">
93      <label><strong>Ask EduGenie a Question:</strong></label><br><br>
94      <input type="text" id="question" placeholder="Why is the sky blue?" required>
95      <button type="submit">Get Answer</button>
96    </form>
97    <div id="qaResult" class="output"></div>
98
99    <!-- Explanation -->
100   <form id="explainForm">
101     <label><strong>Need an Explanation?</strong></label><br><br>
102     <input type="text" id="topic" placeholder="Photosynthesis" required>
103     <button type="submit">Explain</button>
104   </form>
105   <div id="explanationResult" class="output"></div>
106
107   <!-- Summary -->
108   <form id="summaryForm">
109     <label><strong>Summarize a Paragraph:</strong></label><br><br>
110     <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
111     <button type="submit">Summarize</button>
112   </form>
113   <div id="summaryResult" class="output"></div>
114
115   <!-- Quiz -->
116   <form id="quizForm">
117     <label><strong>Generate a Quiz:</strong></label><br><br>
118     <input type="text" id="quizText" placeholder="Solar System" required>
119     <button type="submit">Generate Quiz</button>
120   </form>
121   <div id="quizResult" class="output"></div>
```

Fig. Showing Frontend Intergration

Activity 3.2: Live Integration

- Form submission sends POST request to FastAPI backend
- Result appears below input box in real-time

MILESTONE 4: Deployment

- **Activity 4.1:** Run Locally

Run locally with `uvicorn main:app --reload`

Navigate to: <http://127.0.0.1:8000>

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python3.11 + v [ ] [ ] ... ^ x
```

```
(base) telladivvyareddy@telas-MacBook-Air edugenie % uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/telladivvyareddy/Desktop/edugenie']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17749] using StatReload
/opt/anaconda3/lib/python3.11/site-packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is disabled. Please consider upgrading optree using 'python3 -m pip install --upgrade 'optree>=0.13.0''.
  warnings.warn(
INFO: Started server process [17751]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Fig. Showing the Application can be accessed

- **Activity 4.2: Functional Testing**

Ask a question

Get an explanation

Generate a quiz

Summarize content

Get a personalized learning plan

Welcome to EduGenie 🧠✨

Your personal AI tutor for learning support!

Ask EduGenie a Question:

Get Answer

Need an Explanation?

Explain

Summarize a Paragraph:

Summarize

Fig. EDUGENIE

Summarize a Paragraph:

Paste long content to summarize

Summarize

Generate a Quiz:

Solar System

Generate Quiz

Get Learning Recommendations:

e.g. Linear Regression

Get Recommendations

Fig. EDUGENIE

Exploring EduGenie:

- a. Asking questions:

Welcome to EduGenie 🧠✨

Your personal AI tutor for learning support!

Ask EduGenie a Question:

which is the largest ocean?

Get Answer

Answer:

The Pacific Ocean is the largest ocean on Earth.

b. Explanation of any topic:

Need an Explanation?

Explanation:

Quantum computing is a type of computing that uses the principles of quantum mechanics to perform certain calculations much faster than traditional computers. It allows computers to perform certain calculations much faster than traditional computers because it uses tiny particles called qubits that can exist in multiple states at the same time. This allows for new kinds of calculations and solving problems that are not possible with traditional computers.

Need an Explanation?

Explanation:

The binary search algorithm is a way to find the index of a target value in a sorted list of numbers. It's like searching for a needle in a haystack. You need to find the number between 0 and 1, and then go back to the beginning of the list to find the target value.

c. Summarising long paragraphs:

Summarize a Paragraph:

Summary:

The Industrial Revolution changed the world by shifting it from farming to factories. New machines, like James Watt's steam engine, made things faster and easier to produce. While this boosted economies, it also created problems like bad working conditions, child labor, pollution, and overcrowded cities. Even with these problems, the Industrial Revolution shaped the modern world, affecting how we travel, communicate, work, and trade.

d. Generating quizzes:

It generates three questions with 4 options each. It corrects the answer if chosen wrong.

Generate a Quiz:

Pythagoras theorem

Generate Quiz

Quiz:

Q1: What does the Pythagorean theorem describe?

- ☐ The relationship between the angles of a triangle.
- ☐ The relationship between the sides of a right-angled triangle.
- ☐ The relationship between the area and perimeter of a triangle.
- ☐ The relationship between the sides of any triangle.

Check Answer

Quiz:

Q1: What does the Pythagorean theorem describe?

- ☐ The relationship between the angles of a triangle.
- ☒ The relationship between the sides of a right-angled triangle.
- ☐ The relationship between the area and perimeter of a triangle.
- ☐ The relationship between the sides of any triangle.

Check Answer

✓ Correct!

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

- ☐ $a + b = c$

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

- ☐ $a + b = c$
- ☐ $a^2 + b^2 = c^2$
- ☒ $a^2 - b^2 = c^2$
- ☐ $2a + 2b = 2c$

Check Answer

✗ Incorrect. Correct answer: $a^2 + b^2 = c^2$

Q3: Which type of triangle does the Pythagorean theorem apply to?

- ☐ Equilateral triangles
- ☐ Isosceles triangles
- ☒ Right-angled triangles
- ☐ All types of triangles

Check Answer

✓ Correct!

e. Guiding with learning recommendations from beginners level to advanced level

Get Learning Recommendations:

SQL

Get Recommendations

Learning Recommendations for "SQL":

SQL Learning Path: From Zero to Hero

This learning path is structured to progressively introduce SQL concepts, starting from the basics and gradually advancing to more complex topics. It's designed to be adaptive: feel free to adjust the pace and delve deeper into areas that particularly interest you.

****I. Beginner Level: Building a Foundation****

****(Estimated Time: 1-2 weeks)****

*** **Key Topics:****

- * What is a Database and SQL? (Relational Model, DBMS)
- * Basic Syntax (SELECT, FROM, WHERE)
- * Data Types (INT, VARCHAR, DATE, etc.)
- * Filtering Data (WHERE clause with comparison operators, logical operators: AND, OR, NOT)
- * Ordering Results (ORDER BY)
- * Limiting Results (LIMIT/OFFSET or TOP)
- * Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
- * Introduction to Tables and Columns

*** **Resources:****

*** **Interactive Tutorials:****

- * SQLZoo: Excellent for hands-on practice with different SQL flavors.
- * Khan Academy's SQL course: Offers a good introductory overview.
- * Codecademy's Learn SQL: Interactive lessons with practical exercises.

*** **Videos:****

- * FreeCodeCamp's SQL Tutorial: Comprehensive and beginner-friendly.

****II. Intermediate Level: Working with Multiple Tables****

****(Estimated Time: 2-3 weeks)****

```

**II. Intermediate Level: Working with Multiple Tables**

**(Estimated Time: 2-3 weeks)**

* **Key Topics:**
  * Joining Tables (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN)
  * Subqueries (Nested queries)
  * Grouping Data (GROUP BY)
  * Filtering Groups (HAVING)
  * Set Operations (UNION, INTERSECT, EXCEPT)
  * Working with Strings (string concatenation, substrings, LIKE operator)
  * Working with Dates and Times (date formatting, date arithmetic)
  * Views (Creating and using views)

* **Resources:**
  * **Books:**
    * "SQL Queries for Mere Mortals" by Michael J. Hernandez: A great resource for understanding practical SQL querying.
    * **Online Documentation:** Refer to the documentation for your specific SQL database (e.g., MySQL, PostgreSQL, SQL Server).
    * **Practice Platforms:** LeetCode, HackerRank, and similar platforms offer SQL challenges to reinforce your skills.

**III. Advanced Level: Mastering Database Management**

**(Estimated Time: 3-4 weeks and beyond)**

* **Key Topics:**
  * Stored Procedures and Functions
  * Triggers
  * Indexes and Performance Tuning
  * Transactions and Concurrency Control
  * Database Design (Normalization)

  * **Books:**
    * "SQL Performance Explained" by Markus Winand: In-depth coverage of SQL performance optimization.
    * "Database Internals" by Alex Petrov: Explains the inner workings of database systems.
    * **Advanced Online Courses:** Look for specialized courses on database administration and performance tuning on platforms like Udemy and Coursera.
    * **Community Forums and Blogs:** Engage with the SQL community to learn from experienced professionals and stay up-to-date with the latest advancements.

**Adaptive Learning Tips:**

* **Start with the basics:** Don't rush into advanced topics before mastering the fundamentals.
* **Practice regularly:** The more you practice writing SQL queries, the better you'll become.
* **Use real-world datasets:** Working with real data will help you understand how SQL is used in practical scenarios.
* **Focus on understanding the concepts:** Don't just memorize syntax; understand the underlying logic behind SQL queries.
* **Break down complex problems:** Divide large problems into smaller, manageable parts.
* **Don't be afraid to experiment:** Try different approaches and see what works best.
* **Seek help when needed:** Don't hesitate to ask for assistance from online communities, forums, or mentors.

This learning path provides a solid foundation for mastering SQL. Remember to adapt it to your own learning style and goals, and enjoy the journey of becoming proficient in this powerful language!

```

It provides along with details of resources from where to learn and stepwise guidance through-out.

Conclusion:

EduGenie has been developed as a robust and accessible AI-powered educational assistant that seamlessly integrates cloud-based intelligence with an intuitive, low-footprint web interface. Designed to support self-learners, educational institutions, and content platforms, EduGenie democratizes learning by simplifying

complex concepts, providing instant question-and-answer interactions, generating personalized quizzes, offering detailed summaries, and tailoring smart learning paths based on individual needs. Its lightweight infrastructure ensures that even users with minimal hardware resources can access high-quality, personalized education without barriers.

Throughout the development of EduGenie, the project achieved key milestones in AI integration, user experience design, and content adaptability. Leveraging advanced generative AI, it transforms static learning into a dynamic and interactive process. The platform's simplicity and clarity cater to learners of all levels—especially those who may find traditional educational content overwhelming or inaccessible. The modularity of the platform further allows for easy future upgrades, while its cloud-based backend ensures scalability and data-driven intelligence.

However, the development journey came with its own set of challenges. Ensuring accuracy in AI-generated responses, maintaining performance across devices, and balancing personalization with general usability were significant hurdles. Overcoming these required continuous testing, fine-tuning AI prompt structures, optimizing the UI for clarity, and addressing data privacy concerns. Yet, these challenges provided valuable learning outcomes—especially in areas like prompt engineering, cloud integration, user-centric design, and scalable architecture.

Looking ahead, EduGenie holds immense potential for enhancement and expansion. One promising direction involves the integration of voice-based interaction, enabling users to learn hands-free using spoken commands and queries—a particularly useful feature for accessibility and multitasking. Multilingual support is another key frontier, helping bridge language barriers and reaching a broader global audience. A mobile application is also envisioned to make learning available anytime, anywhere, with offline features for uninterrupted access.

Furthermore, future iterations of EduGenie could incorporate progress tracking dashboards, allowing users to visualize their learning journey through metrics and insights. The addition of gamified elements such as badges, learning streaks, and challenge-based assessments can boost engagement and motivation. Adaptive learning paths powered by data analytics can guide users more intelligently through content based on their strengths and weaknesses.

From a collaboration perspective, EduGenie can evolve to support group study sessions, teacher or parent dashboards, and integration with Learning Management Systems (LMS) such as Moodle or Google Classroom. Input recognition for images and PDF files can expand how students interact with content, enabling snapshot-based doubt solving and resource summarization. Real-

time sync and smart notifications will also improve continuity and interactivity.

In conclusion, EduGenie is not just an AI tool—it's a foundation for the future of personalized, inclusive, and intelligent education. By bridging gaps in access, comprehension, and adaptability, it redefines the learning experience for digital natives and underserved communities alike. As it continues to evolve, EduGenie is poised to become a comprehensive learning companion for the modern era—smart, scalable, and learner-first.

Submitted by:

Tella Divya Sree

Mentor: Siri

Date: 11/04/2025